Generative Image Modeling: A Comparative Analysis of Variational Autoencoders and Denoising Diffusion Probabilistic Models on CIFAR-10

1. Introduction

Generative models aim to learn the underlying probability distribution of data in order to generate new samples that resemble the original dataset. Among these models, there are two prominent fundemental models, Variational Autoencoders (VAEs) and Denoising Diffusion Probabilistic Models (DDPMs). Both represent two fundamentally different approaches for generative modeling.

Variational Autoencoders are latent-variable generative models that learn a compressed representation of the data in a continuous latent space. By combining an encoder–decoder architecture with probabilistic regularization through Kullback–Leibler (KL) divergence, VAEs enable efficient training and fast sample generation. However, they often suffer from limitations in sample sharpness due to the reconstruction-based training objective.
In contrast, DDPMs generate data through an iterative denoising process. Starting from pure noise, the model progressively reconstructs a clean image through a learned reverse diffusion process. Although diffusion models are computationally more expensive during both training and generation, they have recently demonstrated better performance in image synthesis due to their ability to better capture complex data distributions.

The objective of this report is to document the implementation of both Variational Autoencoder (VAE) and Denoising Diffusion Probabilistic Model (DDPM) architectures from scratch, train them on a public image dataset, and conduct a comparative evaluation of their generative performance. The models are trained on the CIFAR-10 dataset, and their performance is analyzed through both quantitative metrics, Fréchet Inception Distance (FID) and Inception Score (IS) and qualitative visual inspection of generated samples. Through this report, we aim to highlight the key differences between these two generative modeling approaches in terms of training dynamics, computational cost, and image generation quality.

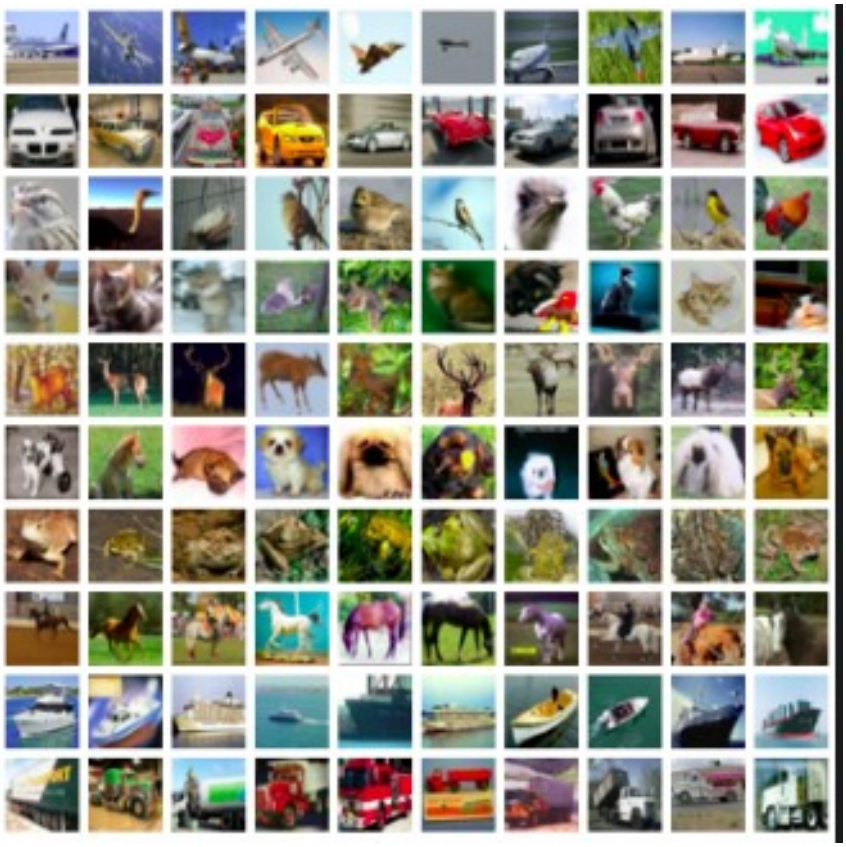
2. Methodology

2.1 Dataset and Experimental Setup

2.1.1 Dataset

CIFAR-10 is a computer-vision dataset. It consists of 60,000 32×32 color images containing one of 10 object classes, with 6,000 images per class. The dataset is split into 50,000 training images and10,000 test images. The 10 classes include airplanes, automobiles, birds, cats, deer, dogs, frogs, horses, ships, and trucks.

Images were normalized to the **ranges [0, 1] for VAE and [-1, 1] for DDPM** during training to maintain numerical stability. No augmentation was applied to the dataset.



A gird of 100 images from CIFAR10 dataset

2.1.2 Hardware and Environment

All experiments were conducted on Kaggle T4 GPU (16GB VRAM) with PyTorch. Development and testing occurred locally on Ubuntu24 with CPU fallback for quick iterations, while final training runs and evaluation were performed on Kaggle's GPU infrastructure to leverage accelerated computation.

The T4 GPU's limited VRAM (16GB) required careful memory management during evaluation, leading to quantitative assessment on 2,000 real images and 500 generated images rather than larger evaluation sets ***(will discussed later in Evaluation section)***

2.2 VAE Implementation

2.2.1 Architecture Design

The VAE architecture follows a symmetric encoder-decoder structure optimized for 32×32 CIFAR-10 images.

The encoder consists of three convolutional layers with ReLU activations, the encoded representation is flattened and passed through two linear layers to produce the mean (μ) and log-variance ($\log \sigma^2$) of the latent distribution, both of dimension 64 or 128. The decoder mirrors this structure: a linear layer projects the latent vector z back to 4,096 dimensions, reshaped to 256 × 4 × 4, followed by three transposed convolutions with ReLU activations and a final Sigmoid activation to map outputs to [0, 1] range.

Notably, I omitted **skip connections** and **batch normalization** despite their common use in autoencoders. Skip connections would bias the model toward reconstruction optimization at the expense of latent space regularization via KL divergence.

Batch normalization was excluded to preserve the statistical properties of the latent dimensions, ensuring the KL term remains a meaningful regularizer.

```
----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
            Conv2d-1           [-1, 64, 16, 16]           3,136
            Conv2d-2            [-1, 128, 8, 8]         131,200
            Conv2d-3            [-1, 256, 4, 4]         524,544
            Linear-4                   [-1, 64]         262,208
            Linear-5                   [-1, 64]         262,208
            Linear-6                 [-1, 4096]         266,240
   ConvTranspose2d-7            [-1, 128, 8, 8]         524,416
   ConvTranspose2d-8           [-1, 64, 16, 16]         131,136
   ConvTranspose2d-9            [-1, 3, 32, 32]           3,075
================================================================
Total params: 2,108,163
Trainable params: 2,108,163
Non-trainable params: 0
----------------------------------------------------------------
Input size (MB): 0.01
Forward/backward pass size (MB): 0.46
Params size (MB): 8.04
Estimated Total Size (MB): 8.52
----------------------------------------------------------------
```

2.2.2 Loss Function and Training

The VAE loss combines reconstruction loss and KL divergence:

$$L = L_recon + \beta \cdot L_KL$$

$$L_KL = -0.5 \cdot \Sigma(1 + \log \sigma^2 - \mu^2 - \sigma^2)$$

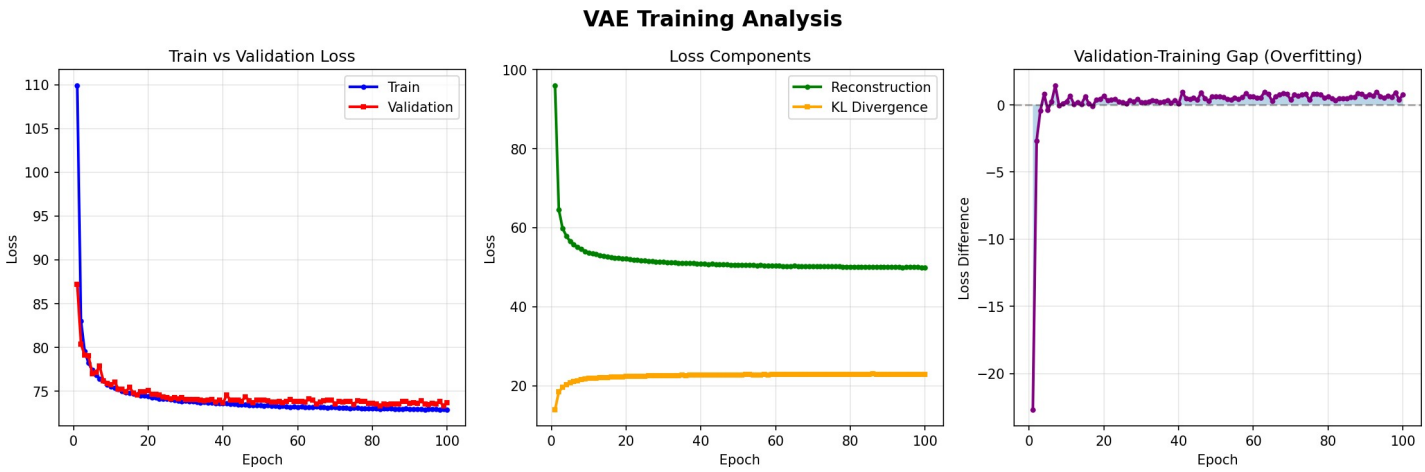
This KL term regularizes the latent space to match a standard Gaussian prior N(0, I), encouraging the model to learn a continuous, well-structured representation. The weight β controls the trade-off between reconstruction fidelity and latent space regularization; in my experiments, I used **β=1.0** to balance both objectives equally.

I conducted two experiments comparing loss functions:

MSE Loss: MSE assumes a Gaussian likelihood p(x|z), which aligns with the Gaussian prior on the latent space. Results show:

- **Train Loss: 72.925**
- **Val Loss: 73.319**
- **KL/Reconstruction Ratio: 0.46**

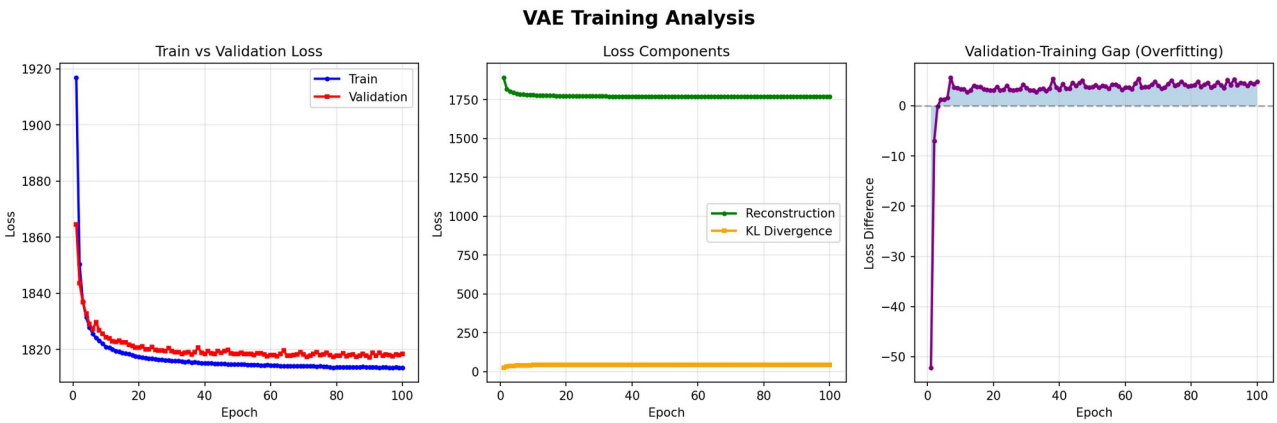
The high KL ratio indicates a well-structured latent space, validating the theoretical alignment between MSE and Gaussian priors. However, this comes at the cost of blurrier reconstructions.



BCE Loss: Assumes Bernoulli likelihood, producing sharper reconstructions with results:

- **Train Loss: 1813.685**
- **Val Loss: 1817.209**
- **KL/Reconstruction Ratio: 0.025**

The dramatic collapse in KL ratio (0.46 → 0.025) stems from the fundamental mismatch we mentioned on the probability distribution used in each loss functions. The difference in absolute loss magnitude (73 vs 1817) is a secondary effect of the different probability distributions. This results is that BCE model assigns minimal probability mass to the KL term and relies almost entirely on reconstruction. Despite this, BCE produces visually superior results indicating that sharp reconstructions are valued more than a structured latent space.



Critical Finding: Using BCE with mean reduction (rather than sum reduction) caused KL collapse, requiring careful loss scaling. I mitigated this by keeping β=1.0 and using sum reduction divided by batch size for final results.

Model Training:

- **Optimizer: Adam (lr=1e-3)**
- **Batch size: 64 - Epochs: 100**
- **Learning rate schedule: Linear**
- **Latent dimensions tested: 64 and 128**

Critical Finding: I conducted experiments with latent dimensions of 64 and 128 to determine optimal representation capacity. Results showed no meaningful difference in either reconstruction quality or latent space structure between the two dimensions. Consequently, I selected latent dimension 64 for all subsequent experiments to reduce computational overhead.

2.3 DDPM Implementation

2.3.1 Architecture Design

The DDPM architecture is built upon a UNet encoder-decoder structure specifically designed for iterative denoising. Unlike the VAE's symmetric design, the UNet incorporates skip connections, multi-scale processing, and attention mechanisms to effectively handle the diffusion process.

The UNet architecture consists of:

- **Initial Convolution: 3 input channels with 64 base channels with a 3×3 kernel, establishing the initial feature representation.**
- **Encoder (Down Blocks): Four progressive downsampling stages with channel multipliers [1, 2, 2, 2], reducing spatial dimensions.**
- **Bottleneck (Middle Block): At the lowest resolution (2×2), two ResNet blocks sandwich an attention block to process the most compressed representation.**
- **Decoder (Up Blocks): Four progressive upsampling stages with skip connections concatenated from corresponding encoder levels.**

- **Output Convolution: Final 3×3 convolution projecting to 3 output channels**
- **(RGB) with GroupNorm and SiLU activation.**

```
==========================================================================================
Layer (type:depth-idx)                   Output Shape              Param #
==========================================================================================
UNet                                     [2, 3, 32, 32]            --
├─TimeEmbedding: 1-1                     [2, 256]                  --
├─Sequential: 1-2                        [2, 256]                  --
│    └─Linear: 2-1                       [2, 1024]                 263,168
│    └─SiLU: 2-2                         [2, 1024]                 --
│    └─Linear: 2-3                       [2, 256]                  262,400
├─Conv2d: 1-3                            [2, 64, 32, 32]           1,792
├─ModuleList: 1-4                        --                        --
│    └─DownBlock: 2-4                    [2, 64, 16, 16]           --
│    │    └─ResNetBlock: 3-1             [2, 64, 32, 32]           90,560
│    │    └─Identity: 3-2                [2, 64, 32, 32]           --
│    │    └─Conv2d: 3-3                  [2, 64, 16, 16]           36,928
│    └─DownBlock: 2-5                    [2, 128, 8, 8]            --
│    │    └─ResNetBlock: 3-4             [2, 128, 16, 16]          263,040
│    │    └─Identity: 3-5                [2, 128, 16, 16]          --
│    │    └─Conv2d: 3-6                  [2, 128, 8, 8]            147,584
│    └─DownBlock: 2-6                    [2, 128, 4, 4]            --
│    │    └─ResNetBlock: 3-7             [2, 128, 8, 8]            328,576
│    │    └─AttentionBlock: 3-8          [2, 128, 8, 8]            66,304
│    │    └─Conv2d: 3-9                  [2, 128, 4, 4]            147,584
│    └─DownBlock: 2-7                    [2, 128, 2, 2]            --
│    │    └─ResNetBlock: 3-10            [2, 128, 4, 4]            328,576
│    │    └─AttentionBlock: 3-11         [2, 128, 4, 4]            66,304
│    │    └─Conv2d: 3-12                 [2, 128, 2, 2]            147,584
├─ResNetBlock: 1-5                       [2, 128, 2, 2]            --
│    └─GroupNorm: 2-8                    [2, 128, 2, 2]            256
│    └─Conv2d: 2-9                       [2, 128, 2, 2]            147,584
│    └─Linear: 2-10                      [2, 128]                  32,896
│    └─GroupNorm: 2-11                   [2, 128, 2, 2]            256
│    └─Conv2d: 2-12                      [2, 128, 2, 2]            147,584
│    └─Identity: 2-13                    [2, 128, 2, 2]            --
├─AttentionBlock: 1-6                    [2, 128, 2, 2]            --
│    └─GroupNorm: 2-14                   [2, 128, 2, 2]            256
│    └─Linear: 2-15                      [2, 4, 128]               16,512
│    └─Linear: 2-16                      [2, 4, 128]               16,512
│    └─Linear: 2-17                      [2, 4, 128]               16,512
│    └─Linear: 2-18                      [2, 4, 128]               16,512
├─ResNetBlock: 1-7                       [2, 128, 2, 2]            --
│    └─GroupNorm: 2-19                   [2, 128, 2, 2]            256
│    └─Conv2d: 2-20                      [2, 128, 2, 2]            147,584
│    └─Linear: 2-21                      [2, 128]                  32,896
│    └─GroupNorm: 2-22                   [2, 128, 2, 2]            256
│    └─Conv2d: 2-23                      [2, 128, 2, 2]            147,584
│    └─Identity: 2-24                    [2, 128, 2, 2]            --
├─ModuleList: 1-8                        --                        --
│    └─UpBlock: 2-25                     [2, 128, 4, 4]            --
│    │    └─ConvTranspose2d: 3-13        [2, 128, 4, 4]            262,272
│    │    └─ResNetBlock: 3-14            [2, 128, 4, 4]            589,184
│    │    └─AttentionBlock: 3-15         [2, 128, 4, 4]            66,304
│    └─UpBlock: 2-26                     [2, 128, 8, 8]            --
│    │    └─ConvTranspose2d: 3-16        [2, 128, 8, 8]            262,272
│    │    └─ResNetBlock: 3-17            [2, 128, 8, 8]            589,184
│    │    └─AttentionBlock: 3-18         [2, 128, 8, 8]            66,304
│    └─UpBlock: 2-27                     [2, 128, 16, 16]          --
│    │    └─ConvTranspose2d: 3-19        [2, 128, 16, 16]          262,272
│    │    └─ResNetBlock: 3-20            [2, 128, 16, 16]          589,184
│    │    └─Identity: 3-21               [2, 128, 16, 16]          --
│    └─UpBlock: 2-28                     [2, 64, 32, 32]           --
│    │    └─ConvTranspose2d: 3-22        [2, 64, 32, 32]           131,136
│    │    └─ResNetBlock: 3-23            [2, 64, 32, 32]           135,888
│    │    └─Identity: 3-24               [2, 64, 32, 32]           --
├─Sequential: 1-9                        [2, 3, 32, 32]            --
│    └─GroupNorm: 2-29                   [2, 64, 32, 32]           128
│    └─SiLU: 2-30                        [2, 64, 32, 32]           --
│    └─Conv2d: 2-31                      [2, 3, 32, 32]            1,731
==========================================================================================
Total params: 5,589,635
Trainable params: 5,589,635
Non-trainable params: 0
Total mult-adds (Units.GIGABYTES): 1.38
```

Critical architectural choices in the architecture:

GroupNorm over BatchNorm: I use GroupNorm(32, channels) instead of BatchNorm throughout the network. GroupNorm computes statistics within groups of channels rather than across the batch, providing stable normalization even with small batch sizes and enabling training at higher batch sizes without performance degradation. This is particularly important for diffusion models, which can be memory-intensive and prevents large batch size training.

SiLU Activation: All nonlinearities use SiLU (Sigmoid Linear Unit) instead of ReLU. SiLU's smooth, self-gating curve better matches the continuous nature of the denoising process and improves model capability for capturing complex noise patterns.
Time Embedding: Timestep information is encoded using sinusoidal position embeddings (as in Transformers), projecting scalar timesteps to a 256-dimensional vector. This is then transformed through a MLP and injected into each ResNet block, enabling the model to learn timestep-dependent denoising strategies.
Attention Blocks: Self-attention is applied at the two deepest resolution levels (4×4 and 2×2) in both encoder and decoder, with each attention block containing 66,304 parameters. These capture long-range dependencies in the noise prediction task.
The complete UNet contains **5,589,635 trainable parameters**, substantially larger than the VAE with **2,108,163 parameters** due to the added complexity of attention mechanisms and skip connections across multiple scales.

2.3.2 Noise Schedule

A critical component of diffusion models is the noise schedule and during the process of siffusion, there are two aspects of noise that should be considered:

- The amount of noise that should be added in each time step, the noise curve.
- The amount of noise to be added back after subtracting noise at a given timestep.

I implement the cosine scheduler that is more helpful than the linear noise addition, since the cosine addition will help the model to take its time in picking the contours and the details of the image, while the linear addition would force the model to start learning right from the very first timestep.
The cosine noise scheduler defined as:

$$\bar{\alpha}_t = \cos^2((t/T + s) / (1 + s) \cdot \pi/2)$$

where $t \in [0, T]$, T is the total number of timesteps (400 and 1000 in my experiments), and $s = 0.008$ is a small constant for numerical stability. The cumulative product $\bar{\alpha}_t$ governs how much of the original signal remains at timestep t, while $\beta_t = 1 - (\bar{\alpha}_t / \bar{\alpha}_{t-1})$ determines the variance of noise added.

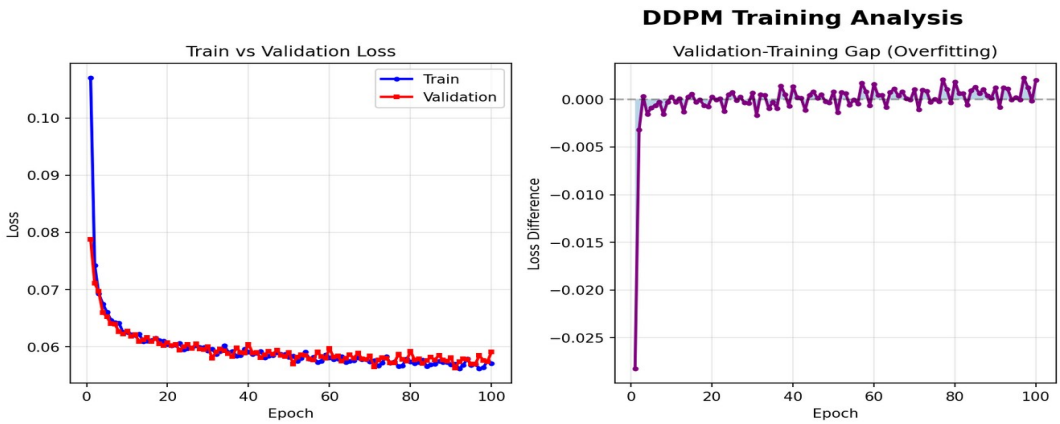
2.3.3 Training Experiments

I conducted experiments comparing timestep counts: 400 vs 1000. The results show that **timestep count has NO meaningful effect on training.**
400 Timesteps:

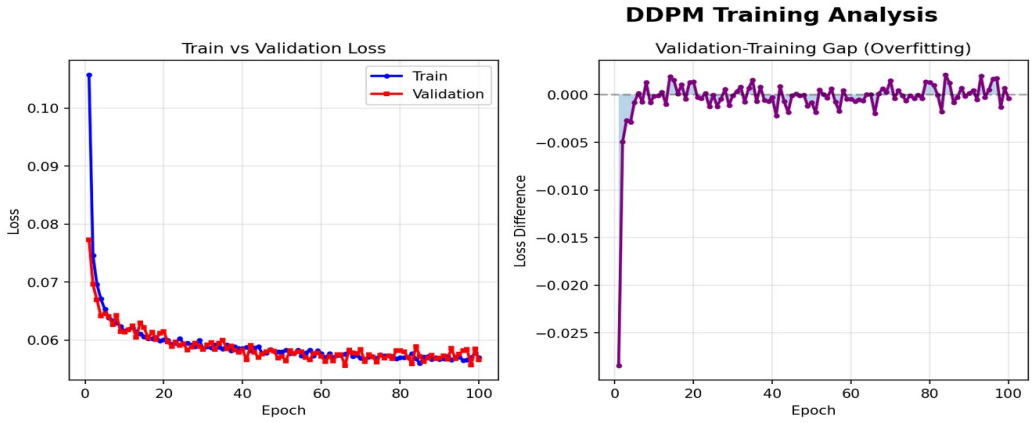
- **Final Train Loss: 0.0576**
- **Final Val Loss: 0.0556**
- **Avg Epoch Time: 74.2s**
- **Total Time: 123.6 minutes**

1000 Timesteps:

- **Final Train Loss: 0.057**
- **Final Val Loss: 0.0562**
- **Avg Epoch Time: 75.5s**
- **Total Time: 125.9 minutes**



Loss function for the experiment with 1000 timesteps



Loss function for the experiment with 400 timesteps

The loss curves are nearly identical, confirming that timesteps primarily affect inference speed, not training dynamics. This is because during training, timesteps are uniformly sampled, so the model learns to denoise at all scales regardless of schedule granularity. However, timesteps critically affect generation speed as the model has a generation speedup of 2.7x faster with 400 timesteps

Generation Time Comparison (500 images):

- **1000 timesteps: 4.6 minutes (7.3 sec/image)**
- **400 timesteps: 1.7 minutes (3.0 sec/image)**

This speedup is achieved through GPU parallelism: while individual images take longer (3 sec vs 7 sec), generating a batch of 16 images takes only 3.6 sec (not 16 × 3 = 48 sec) due to GPU parallelization of forward passes. **The effective per-image cost in batches is much lower than I initially thought (sequential inference).**

2.3.4 Training Configurations and Loss Functions Formula

Model Training:

- **Optimizer: Adam**
- **Batch size: 64**
- **Epochs: 100**
- **Learning rate schedule: Cosine annealing from 2e-4 to 1e-6**
- **Loss function: MSE between predicted and actual noise**

The training loss (MSE) ***(L = ||noise_pred - noise_actual||²_2)*** directly optimizes the model's ability to predict noise in noisy images. Unlike VAE's combined loss (reconstruction + KL), DDPM has a single, straightforward objective.

Critical Implementation: Sampling strategy during Generation and Bug Fix

The sampling (reverse diffusion) process is where DDPM's true computational cost emerges. Starting from pure noise, the model iteratively denoises over 400 or 1000 steps, each requiring a forward pass through the UNet.

Initial sampling implementation produced severely degraded images with pixel values **exploding beyond the valid [-1, 1] range, with observed ranges of [-826, 787]**.

Investigation revealed the issue: naive removal of predicted noise followed by re-injection of noise using β (rather than the posterior variance β_tilde) caused numerical instability. The fix involved two critical corrections:

1. x_0 Reconstruction with Clipping: Instead of directly modifying x_t, we first reconstruct the predicted clean image: $x_0_pred = (x_t - \sqrt{(1 - \bar{\alpha}_t)} \cdot noise_pred) / \sqrt{(\bar{\alpha}_t)}$
   Then clamp to the valid range: $x_0_pred = clip(x_0_pred, -1, 1)$.
   This ensures intermediate predictions stay bounded and prevents error accumulation across 400 or 1000 steps.
2. Posterior Variance β_tilde: The reverse diffusion process should use the posterior variance, not β: $\beta_tilde = (1 - \bar{\alpha}_\{t-1\}) / (1 - \bar{\alpha}_t) \cdot \beta_t$ as using β directly injects excessive noise that undoes the denoising progress.

3. RESULTS & QUANTITATIVE EVALUATION

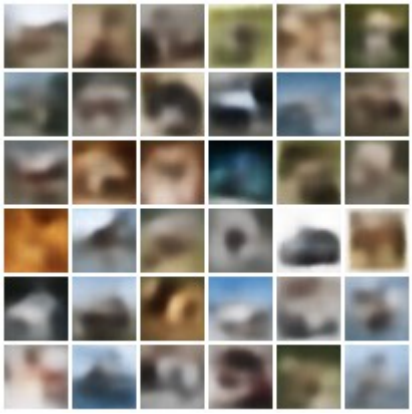
3.1 VAE Results

3.1.1 VAE with MSE Loss

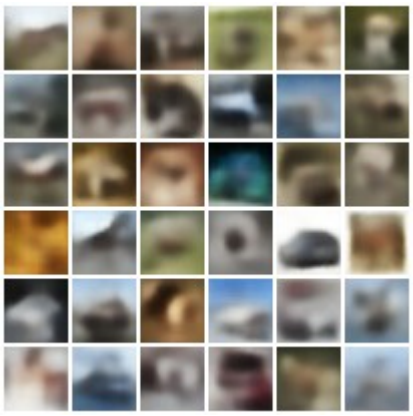
The VAE trained with MSE quantitative evaluation:

- **FID Score: 206.19**
- **Inception Score: 2.84 ± 0.36**

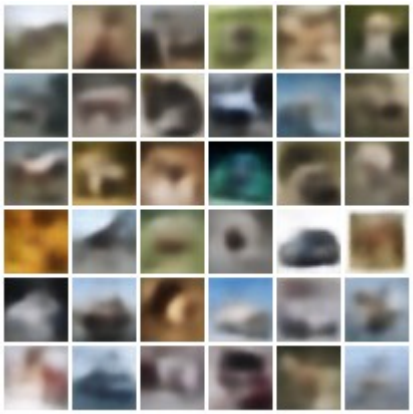
The FID score of 206.19 indicates substantial distribution distance from real CIFAR-10 images, while the IS of 2.84 suggests limited ability to generate recognizable, diverse objects.



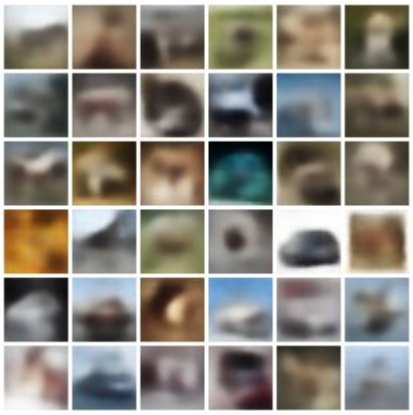
Epoch 20



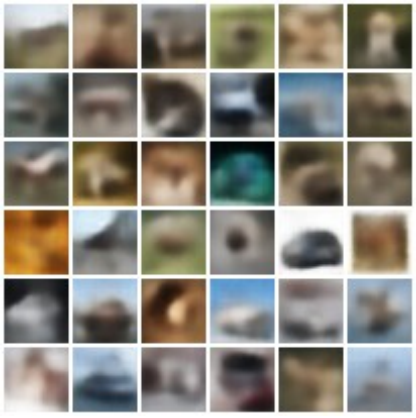
Epoch 40



Epoch 60



Epoch 80



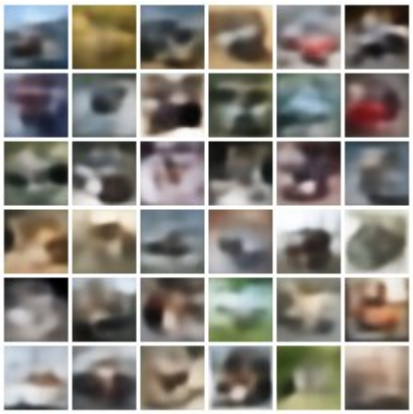
Epoch 100

3.1.2 VAE with BCE Loss

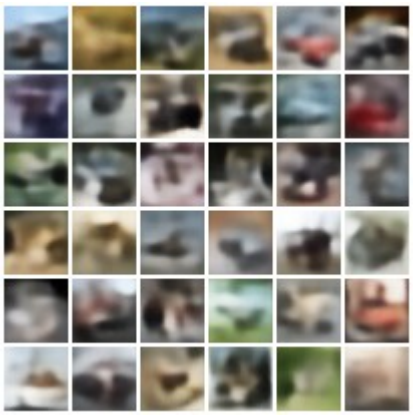
The VAE trained with BCE quantitative evaluation:

- **FID Score: 243.01**
- **Inception Score: 2.30 ± 0.16**

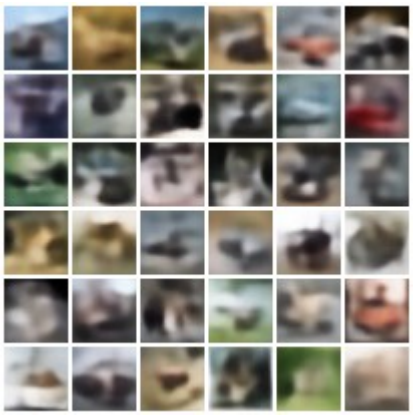
Despite the KL collapse, BCE produced sharper visual reconstructions **(subjectively)**, yet the quantitative metrics (higher FID, lower IS) suggest worse performance than MSE. This could raise another argument that BCE optimizes for pixel-level sharpness at the expense of learning a meaningful, generalizable distribution.



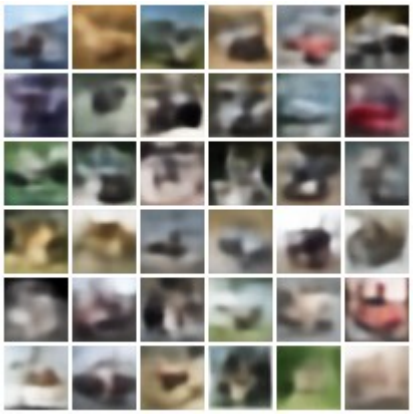
Epoch 20



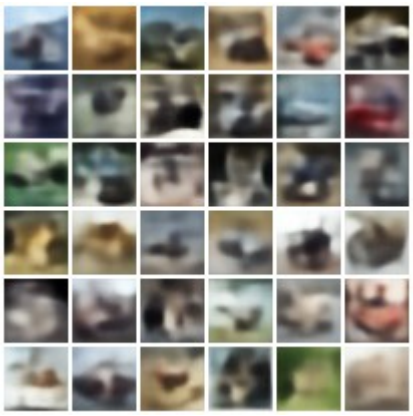
Epoch 40



Epoch 60



Epoch 80



Epoch 100

3.1.3 VAE Key Observations

Both VAE variants converge to the same training speed **30 minutes** for **100 epochs** confirming that the loss function choice does not affect computational cost. The MSE variant's superior quantitative metrics (FID 206 vs 243, IS 2.84 vs 2.30) indicate better generalization, validating the theoretical alignment between Gaussian distribution and Bernoulli distribution.

Generated samples from both VAEs exhibit severe blur and loss of fine detail, consistent with the well-known limitations of pixel-wise reconstruction losses in autoencoders. The generated images show no coherent texture, anatomy, or realistic composition.

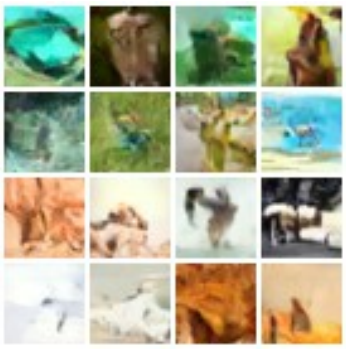
3.2 DDPM Results

3.2.1 DDPM with 400 Timesteps

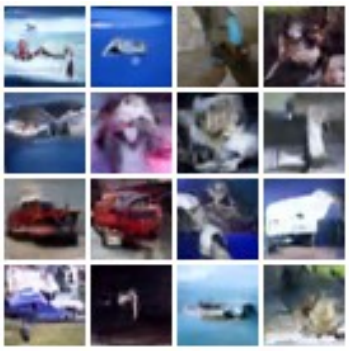
DDPM with 400 timesteps quantitative evaluation:

- **FID Score: 94.73**
- **Inception Score: 4.13 ± 0.34**
- **Generation Time: 1.7 minutes for 500 images (3.0 sec/image)**

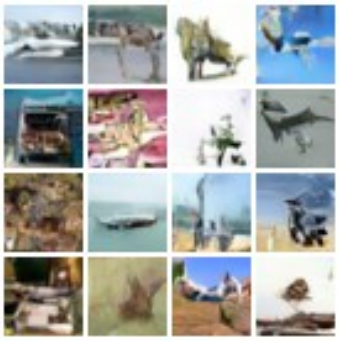
The FID score of 94.73 represents a dramatic improvement over VAE, indicating generated samples are much closer to the real CIFAR10 distribution. The IS of 4.13 shows substantially improved object recognizability and diversity compared to VAE variants.



Epoch 20

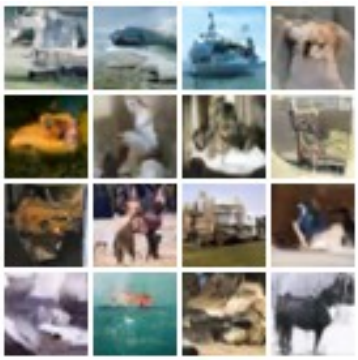


Epoch 40



Epoch 60

Epoch 80



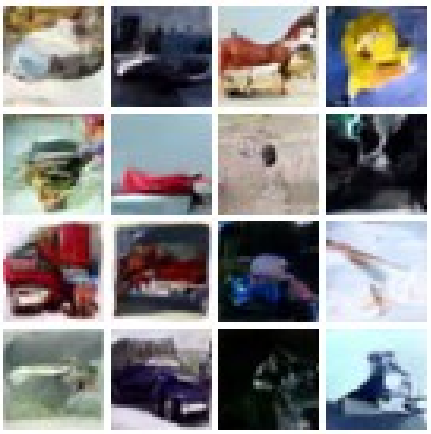
Epoch 100

3.2.2 DDPM with 1000 Timesteps

DDPM with 400 timesteps quantitative evaluation:

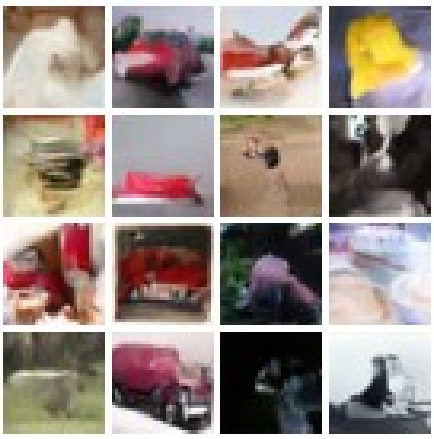
- **FID Score: 75.16**
- **Inception Score: 4.37 ± 0.46**
- **Generation Time: 4.6 minutes for 500 images (7.3 sec/image)**

The FID score of 75.16 represents a 20% improvement over the 400 timestep variant, demonstrating that additional denoising steps during sampling produce noticeably higher quality. The IS of 4.37 is also higher, indicating improved diversity and clarity.

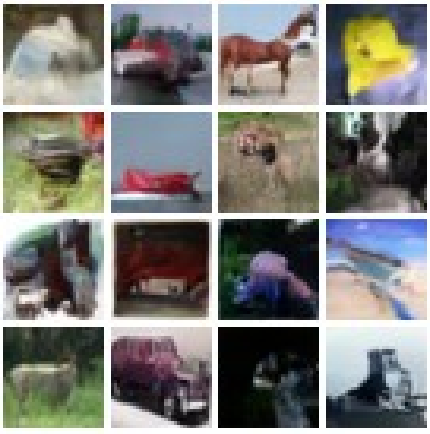


Epoch 20

Epoch 40



Epoch 60



Epoch 80

Epoch 100

3.2.3 DDPM Key Observations

The timestep comparison reveals the fundamental insight of diffusion models: training is insensitive to schedule granularity but sampling quality scales directly with step count. The 2.7× generation speedup (1.7 min vs 4.6 min for 500 images) with 400 timesteps comes at a measurable but acceptable quality cost (FID improvement of 20% with 1000 steps).

As mentioned earlier, GPU parallelism enables efficient batch generation: while per-image generation takes 3–7 seconds depending on timesteps, batching 16 images takes only 3.6 seconds total with 400 timesteps and 8.7 seconds with 1000 timesteps, yielding an effective per-image cost much lower than sequential inference suggests of sequential generation.

3.3 Evaluation Methodology and Limitations

Quantitative metrics (FID and Inception Score) were computed on 500 generated images and 2,000 real CIFAR-10 test images, rather than the typical 10,000 real / 5,000 generated benchmark. This reduction was necessary due to GPU memory constraints (16GB T4) and ensures fair comparison across all four model variants.
The smaller evaluation set may underestimate absolute scores but provides reliable relative comparisons, as all models are evaluated on identical data with identical metrics.

Model	FID	IS	Generation Time	Training Time
VAE (MSE)	**206.19**	**2.84 ± 0.36**	**Milliseconds**	**30 min**
VAE (BCE)	**243.01**	**2.30 ± 0.16**	**Milliseconds**	**30 min**
DDPM (400 steps)	**94.73**	**4.13 ± 0.34**	**1.7 min**	**123.6 min**
DDPM (1000 steps)	**75.16**	**4.37 ± 0.46**	**4.6 min**	**125.9 min**

Summary Table: Quantitative Results

Key Observations:

- **DDPM substantially outperforms VAE on both metrics**
- **1000 timesteps provides 20% FID improvement over 400 steps**
- **VAE training is 4× faster than DDPM**
- **DDPM generation with 400 steps is 2.7× faster than 1000 steps**

4. QUALITATIVE ANALYSIS

4.1 Diversity Assessment

DDPM variants (both 400 and 1000 timesteps) demonstrate exceptional diversity throughout training. Generated samples show a wide, balanced mix of CIFAR10 classes like animals (dogs, cats, horses, sheep, birds), vehicles (trucks, cars, airplanes, ships), and natural scenes. Importantly, within-class variation is evident—different poses, angles, lighting, colors, and textures appear across samples, indicating the model has learned a multi-modal distribution.

VAE variants show only superficial diversity. While the fixed grids display a spread of colors and rough shapes, the extreme blur makes it impossible to confidently identify distinct object classes or poses. The apparentmvariation likely reflects color noise rather than semantic diversity across recognizable categories.

Diversity Ranking: DDPM 1000 ≥ DDPM 400 >> VAE variants

4.2 Realism and Detail Fidelity

DDPM 1000 Timesteps:

Generated samples show clear progression over epochs. At epoch 20, outputs are noisy but contain discernible structure. By epoch 60–80, recognizable objects with basic textures emerge. At epoch 100, many samples demonstrate convincing realism: fur texture on animals, metallic sheen on vehicles, water reflections, grass detail, realistic lighting, and shadows. Fine details and anatomical correctness are largely preserved.

DDPM 400 Timesteps:

Reaches high quality by epoch 100, with good proportions, coherent composition, and reasonable texture. However, fine edges show slightly more blur, occasional oversmoothing, and minor lingering noise artifacts compared to 1000 steps. Still far superior to any VAE output and acceptable for practical generation tasks.

VAE Variants (MSE and BCE):

Both converge almost immediately to heavily blurred, featureless mush with no edges, textures, correct anatomy, or coherent composition. No meaningful improvement occurs beyond epoch ~20. The MSE variant appears marginally less uniform but remains equally unusable. Classic pixel-wise reconstruction artifacts (over-smoothing, mode averaging) completely dominate both VAE outputs.

Realism Ranking: DDPM 1000 > DDPM 400 >> VAE variants

4.3 Thematic Consistency

DDPM variants maintain excellent class-specific coherence throughout training. Every sample remains strictly within the learned CIFAR10 distribution, horses look like horses, airplanes like airplanes, ships like ships, etc.
Class proportions appear well-preserved, reflecting the training distribution even in early noisy epochs. No out-of-distribution drift or semantic collapse is observed.

VAE variants show weak and ambiguous thematic consistency. While color distributions remain roughly stable, the blur prevents reliable verification of whether the model is actually generating intended themes (animals vs. vehicles) or merely producing generic smoothed blobs.
Thematic Consistency Ranking: DDPM 1000 ≥ DDPM 400 >> VAE variants

4.4 Overall Visual Assessment

DDPM represents a fundamental advance in generative modeling. Both variants (even with only 400 denoising steps) produce samples that are visually plausible, diverse, and thematically coherent. The iterative denoising process allows the model to learn complex, multi-modal distributions without collapsing to blurry averages.

VAE represents the limitations of pixel-wise reconstruction losses. Both variants (MSE and BCE) suffer from severe over-smoothing, mode averaging, and complete loss of detail. The learned representations fail to capture the sharp, structured nature of natural images, making the generated samples unsuitable for practical use.

The 1000-timestep DDPM consistently produces the highest visual quality and represents the optimal choice when sample fidelity is paramount. The 400-timestep variant offers an excellent practical compromise, sacrificing ~20% FID improvement in exchange for 2.7× faster generation.